

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 4

4 Prolog (ijl20)

In your answers ensure each relation has a comment giving a declarative reading of its behaviour. Avoid unnecessary use of *cut* and do not use extra-logical relations such as `findall`, `assertz` and `not (\+)`. Built-in library relations should not be assumed. The Prolog operator in `A\=B`, meaning A will not unify with B, may be used.

Answer: Note: use of `cut` will be -1 mark for each part where it is used.

unification,
occurs check

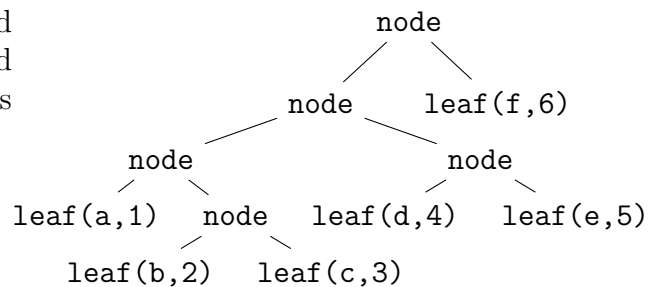
- (a) Explain Prolog's process of *unification*. What situation would an *occurs check* guard against? [3 marks]

Answer: 1. A *variable* will unify with anything. 2. A *compound term* will unify with another term with the same functor and arity if each of the arguments will unify. 3. An *atom* will unify with the same atom (or a variable). 4. The occurs guards against an *infinite loop* occurring if an attempt is made to unify a variable with a compound term containing the *same variable*.

2 marks for 1 to 3, drop a mark if one wrong. 1 mark for 4.

data
representation,
tree search

- (b) Assume `node(Left,Right)` and `leaf(Name,Value)` compound terms are used to represent trees such as:



Define a relation `lookup(+Tree,?Name,?Value)` which finds the value(s) associated with a given name in trees of the above form. [3 marks]

Answer: The compound term defining the tree is not required in the answer, but can be assumed e.g. as in the following fact:

```

tree(node(
    node(
        node(
            leaf(a,1),
            node(
                leaf(b,2),
                leaf(c,3)
            )
        ),
        node(
            leaf(d,4),
            leaf(e,5)
        )
    ),
    leaf(f,6)
))
  
```

).

The definition of `lookup` is as below:

```
% lookup(+Tree,?Name,?Value) succeeds if a leaf term containing the matching
% Name and Value can be found in Tree.
lookup(leaf(Name,Value), Name, Value).
lookup(node(L,_), Name, Value) :- lookup(L,Name,Value).
lookup(node(_,R), Name, Value) :- lookup(R,Name,Value).
```

Declarative comment 1 mark. Correct base case for `leaf` 1 mark, Correct recursive cases and 3-clause solution 1 mark.

lists,
accumulators,
operators,
arithmetic

- (c) Given a list of atoms, `L1`, define a relation `rle(+L1,?L2)` which run-length *encodes* `L1` into `L2`. For example, `rle([a,a,b,c,a,a,a],L)` should succeed with `L=[2*a, 1*b, 1*c, 3*a]`. Giving reasons, indicate for your answer whether a query `rle(L,[2*a, 1*b, 1*c, 3*a])` would succeed with `L=[a,a,b,c,a,a,a]`. [5 marks]

Answer:

```
% rle(+L1, ?L2) succeeds if list L2 is input list L1 run-length encoded
rle(L1,L2) :- rle_acc(L1,L2,1).
```

```
% rle_acc(+L1, ?L2, +N) succeeds if list L2 is input list L1
% run-length-encoded, assuming input N is the accumulated count
% so far of the head of L1.
rle_acc([A],[N*A],N).
rle_acc([A,A|L1],L2,N) :- N1 is N+1, rle_acc([A|L1],L2,N1).
rle_acc([A,B|L1],[N*A|L2],N) :- A \= B, rle_acc([B|L1],L2,1).
```

Declarative comments 1 mark. Use of accumulator 1 mark. Correct base case 1 mark. Correct `rle_acc(+L1,?L2,+N)` relation using 3 clauses 1 mark. Explain is requires ground RHS expression 1 mark.

lists, operators,
arithmetic

- (d) Complementary to `rle/2`, define a relation `rld(+L1,?L2)` which *decodes* a run-length-encoded list `L1` as defined in part (c) into `L2`. [4 marks]

Answer:

```
% rld(+L1,?L2) succeeds if L2 is the decoded list of atoms from
% run-length-encoded list L1.
rld([],[]).
rld([1*X|L], [X|M]) :- rld(L,M).
rld([N*X|L], [X|L2]) :- N > 1, M is N-1, rld([M*X|L],L2).
```

Declarative comment 1 mark. Working base case 1 mark. Succinct 3 clauses 1 mark. Working solution 1 mark.

Note the lack of the guard `N > 1` ensuring correct backtracking is a likely error losing 1 mark.

lists

- (e) The Prolog relations below, given a query `alter_list([2,4,6],L)`, will succeed with `L=[a,a,b]`. Use an additional difference-list argument to accumulate the execution path through the Prolog clauses. Number the clauses 1 to 4 such that

`alter_list([2,4,6],L,Path-[])` will succeed with the sequence of clauses as a list of integers in Path, i.e. with `L=[a,a,b]` and `Path=[4,1,4,1,4,2,3]`.

```
change(N,a) :- N < 5.  
change(N,b) :- N >= 5.
```

```
alter_list([],[]).  
alter_list([H1|T1],[H2|T2]) :- change(H1,H2), alter_list(T1,T2).
```

[5 marks]

Answer:

```
% change(+N,X,Path-P) will succeed if X=a for input integers N less than 5,  
% X=b for input integer N >= 5, Path is the difference list for the  
% successful clause used with P as the end of that list.
```

```
change_path(N,a, [1|P]-P) :- N < 5.  
change_path(N,b, [2|P]-P) :- N >= 5.
```

```
% alter_path(+L1,?L2, Path-P) will succeed if each element of L2 is the  
% changed value of input elements from L1, Path is the difference list  
% for the successful clauses used with P as the end of that list.
```

```
alter_path([],[], [3|P]-P).  
alter_path([H1|T1],[H2|T2], [4|P1]-P3) :-  
    change_path(H1,H2,P1-P2), alter_path(T1,T2,P2-P3).
```

```
% A convenience relation for testing:
```

```
q_alter(L, Path) :- alter_path([2,4,6,8,10],L, Path-[]).
```

Declarative comment 1 mark. Correct accumulator in the `change_path` relation 1 mark. Correct accumulator in the `alter_path` head term 1 mark. No additional relations needed 1 mark. Correct path generation 1 mark.
